

Figure 1: On-disk layout of a graph-based ANNS index. The directed graph is stored as fixed-size records on disk. Each record contains a vector and its out-neighbor IDs (edges). The maximum out-neighbor number of each node is limited to R .

are scattered along the search path. For concurrency control, simply locking all of them before updating causes near-root lock contention. An alternative approach is to leverage the approximate features of insert and search for better concurrency, but how to do this efficiently on disk is still under-exploited.

To address the challenges, we propose ODINANN, a billion-scale graph-based ANNS index with direct inserts. ODINANN achieves efficient direct insert with two techniques as follows.

First, to reduce disk write overhead, we propose *GC-free update combining*. The key observation is that the graph index is stored as fixed-size records on disk. This enables out-of-place record updates to be performed without garbage collection. Therefore, we do space overprovision on disk to reserve multiple free record slots in a page, where multiple record updates can be combined. The old records can be directly recycled without GC. By default, ODINANN doubles disk space to combine updates, resulting in $2\times$ disk writes compared to a log-structured layout.

Second, to increase the concurrency of insert and search, we introduce *approximate concurrency control*, leveraging the approximate features of operations for better parallelism. The key idea is to ensure per-record isolations rather than per-operation ones. For vector search, only a consistent snapshot for each record is required to ensure its correctness. For vector insert, it links the target vector with an approximate neighbor snapshot. Besides, we propose two optimizations for insert, tailored for disk I/O and computing in the critical sections separately, to further improve the concurrency.

We compare ODINANN with state-of-the-art ANNS indexes supporting updates, including graph-based DiskANN [23] and cluster-based SPFresh [30]. In workloads with concurrent inserts and searches, ODINANN shows stable performance, with a median search latency fluctuation¹ of only $1.07\times$, compared to DiskANN's $2.44\times$. Compared to SPFresh, ODINANN achieves 62.1% median search latency and $\sim 15\%$ higher accuracy simultaneously, because of the intrinsic advantage of graph-based indexes over cluster-based ones. In billion-scale datasets, ODINANN simultaneously reaches 5000 QPS search throughput and 1100 QPS insert throughput, with a consis-

¹ Defined as (maximum value / minimum value) in this paper.

Algorithm 1 Vector Search

```

1:  $G \leftarrow$  graph,  $q \leftarrow$  query vector
2: procedure SEARCH( $G, q$ )
3:    $s \leftarrow$  starting vector,  $l \leftarrow$  candidate pool length
4:   candidate pool  $P \leftarrow \{ \langle s.ID, PQ\_distance(s, q) \rangle \}$ 
5:   explored pool  $E \leftarrow \emptyset$ ,  $i \leftarrow 0$ 
6:   while  $i < l$  do
7:      $i \leftarrow$  index of the first vector in  $P$  but not in  $E$ 
8:      $r \leftarrow$  record with  $ID = P[i].ID$   $\triangleright$  read from disk.
9:      $E.insert(r)$ 
10:    for  $nid$  in  $r.neighbors$  do
11:       $P.insert(\langle nid, PQ\_distance(nid, q) \rangle)$ 
12:    end for
13:     $P \leftarrow l$  nearest vectors to  $q$   $\triangleright$  PQ distance.
14:  end while
15:   $L \leftarrow k$  nearest vectors to  $q$  in  $E$   $\triangleright$  exact distance.
16:  return  $E, L$ 
17: end procedure

```

tently stable median search latency of $\sim 3ms$.

In summary, this paper makes the following contributions:

- We analyze the inefficiency of buffered inserts for on-disk graph-based ANNS indexes (§2).
- We propose ODINANN, a billion-scale graph-based ANNS index with direct inserts. It achieves efficient direct inserts by two key techniques, GC-free update combining, and approximate concurrency control (§3).
- We evaluate ODINANN to show its efficacy in stabilizing the search performance and reducing memory usage during concurrent inserts and searches (§4).

2 Background and Motivation

In this section, we first introduce the layout and operations of on-disk graph-based ANNS indexes. Then, we show the inefficiency of buffered insert and how direct insert overcomes it. Finally, we describe the challenges of direct insert.

2.1 On-Disk Graph-Based ANNS Index

Graph layout. In a graph-based index, vectors are organized as a directed graph, which is stored as adjacent lists on disk, as shown in Figure 1. Each disk page contains several records. Each record consists of a vector (node) and its out-neighbor IDs (edges). To ensure a fixed size for records, the nodes' maximum out-degrees are globally limited to R , where R is a configurable parameter.

Operations. To our knowledge, DiskANN [23] is the only on-disk graph-based ANNS index supporting updates. Here, we introduce its search and buffered insert operations.

Vector search. As shown in Algorithm 1, the search process follows a best-first approach. It starts at a fixed starting vector

Algorithm 2 Vector Insert

```
1:  $G \leftarrow$  graph,  $q \leftarrow$  vector,  $R \leftarrow$  max out-degree of  $G$ 
2: Prune( $N, R$ ): remove items in  $N$  until  $N.size() \leq R$ 
3: procedure INSERT( $G, q$ )
4:    $E \leftarrow$  vectors explored in SEARCH( $G, q$ )  $\triangleright$  neighbors.
5:    $E \leftarrow$  Prune( $E, R$ )
6:   set  $q.neighbors$  to  $E$   $\triangleright$  add edges.
7:   for  $nbr$  in  $E$  do  $\triangleright$  add reverse edges.
8:      $N \leftarrow \{nbr.neighbors, q\}$ 
9:      $N \leftarrow$  Prune( $N, R$ )
10:    set  $nbr.neighbors$  to  $N$ 
11:   end for
12: end procedure
```

s and maintains a candidate pool P containing the nearest vectors to the target vector (sorted by their distances). In each step, it explores one or more nearest unexplored vectors, depending on the beam width. The search terminates when all the vectors in P are explored.

Buffered insert. Algorithm 2 shows the process of inserting a vector q into a graph-based index. It first searches q and records the explored vectors in the search path, which are q 's candidate neighbors. Then, it attempts to add bi-directional edges to q and its candidate neighbors. If an edge set exceeds the out-degree limit, it is pruned according to some pre-defined rules (e.g., triangle inequality [24]).

DiskANN adopts buffered insert, in order to reduce disk I/O for neighbor updates by batching them. Specifically, inserts are first absorbed by an extra in-memory index. When searching, both the in-memory and on-disk indexes are traversed. When the in-memory index size reaches a threshold, a *merge* is triggered, which includes two steps. First, it inserts all the vectors in the in-memory index to the on-disk index using Algorithm 2; all the on-disk graph updates are buffered in memory (we call this step *in-memory merge*). Second, the on-disk updates are committed in batches.

2.2 Buffered Insert is Inefficient

Buffered insert is inefficient when merging to disk. We evaluate DiskANN to show this issue. We concurrently do buffered inserts and multi-thread searches in an index with 100 million vectors in the BIGANN [12] dataset. Merge is triggered when the in-memory index contains 6 million vectors (i.e., 6% of the on-disk index size). More detailed configurations are shown in §4.1. Figures 2(a) and 2(b) show the timeline, in which we find three issues.

Issue #1: fluctuated search performance. As shown in Figure 2(a), the search median latency increases to $1.54\times$ on average during the first merge. This is because, merge needs to search the on-disk index to find the neighbors of the target vector, which causes severe read bandwidth interference with the frontend search tasks.

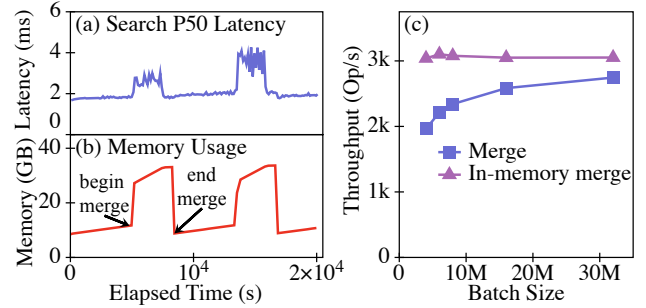


Figure 2: Inefficient buffered insert. (a, b) Concurrent insert and search vectors in an index with 100 million vectors, merge 6 million buffered inserts each time, two merges in total. (c) The throughput of merge increases with the batch size but is bottlenecked by its first step (i.e., in-memory merge).

Issue #2: high memory consumption. As shown in Figure 2(b), when merging 6% (i.e., 6 million) vectors to disk, the peak memory usage exceeds 32GB, more than 50% of the size of the on-disk index with 100 million vectors (~ 55 GB). The in-memory elements include the index and buffered disk updates, whose size linearly increases with the number of buffered inserts. Therefore, larger datasets force more frequent merges due to memory capacity limits. For example, merging 3% of vectors (i.e., 30 million) into a billion-scale index requires 125GB of memory [23].

Issue #3: long merge time. In Figures 2(a) and 2(b), merge accounts for more than 30% of the whole timeline, during which the system suffers from the two issues above. In our configuration, we restrict the insert throughput to 1200 QPS and do not insert vectors during merge. Merge will account for a higher time percentage in workloads with higher insert throughput or serving inserts during the merge.

One may consider increasing the batch size to reduce the time percentage of merge. However, **merge gains little performance boost even with a large batch.** As shown in Figure 2(c), the throughput of merge increases with the batch size, but is bottlenecked by *in-memory merge*, whose throughput remains constant regardless of the batch size. This is because, although disk writes are merged, the *in-memory merge* is still executed *one by one*. For each vector, it searches the on-disk index for neighbors and incurs hundreds of disk reads.

2.3 Opportunity: Direct Insert

We propose to do direct insert instead of buffered insert. Using direct insert, vectors are directly inserted into the on-disk index one by one, instead of buffered in memory and merged to disk later as in buffered insert.

Direct insert can naturally solve the three issues above. First, it evens out the bandwidth interference across the whole timeline, which contributes to stable frontend search per-

formance. Second, it avoids both the in-memory index and buffered disk writes during merge, thus saving memory. Third, it is possible for direct insert to be as efficient as buffered insert, as inserts can not be efficiently batched.

However, it is non-trivial to do direct insert efficiently. There are several challenges as follows.

Challenge 1. *Each insert updates tens or hundreds of neighbor records. Applying the updates in place causes massive random SSD writes, while using a log-structured layout suffers from frequent garbage collection.*

In a graph-based index, each vector connects to tens or hundreds of vectors for navigation [11, 27]. According to Algorithm 2, bi-directional edges are added to the target vector and its neighbors during insert. This process updates the neighbor records scattered randomly on tens or hundreds of pages [27], which leads to massive random SSD writes.

For fewer writes, a straightforward solution is to use a log-structured data layout. However, inserting 10% new records makes the index 12.8× larger (assuming an out-degree of 128); only 7.9% of the records are valid. Thus, garbage collection, which gathers the valid records into a new index, is frequently required. This slows down frontend search requests.

Challenge 2. *Ensuring the isolation of direct insert and search harms their concurrency, due to near-root lock contention. However, the potential of using the approximate index feature for better concurrency is under-exploited.*

According to Algorithm 2, insert may update all the records explored in the search path. To ensure the isolation of insert and search, a naive approach is to lock all of the involved records before updating. However, the record set contains near-root records. Locking them causes serious lock contention, blocking concurrent inserts and searches.

An alternative approach is to leverage the approximate feature of operations for fine-grained concurrency control: Atomicity and isolation of operations are not mandatory for an approximate index, so they can be relaxed for better concurrency. However, how to design an efficient concurrency control protocol for on-disk graph-based indexes, which efficiently utilizes this feature for shorter critical sections and better concurrency, is still under-exploited.

3 Design and Implementation

We design ODINANN, a billion-scale graph-based ANNS index supporting direct inserts and buffered deletes. ODINANN achieves efficient direct insert with the following goals.

- **Reduced disk writes.** Direct insert in a graph index updates tens or hundreds of neighbor records. We do space overprovision to combine these updates into fewer disk writes, while not suffering from GC overhead (§3.2).

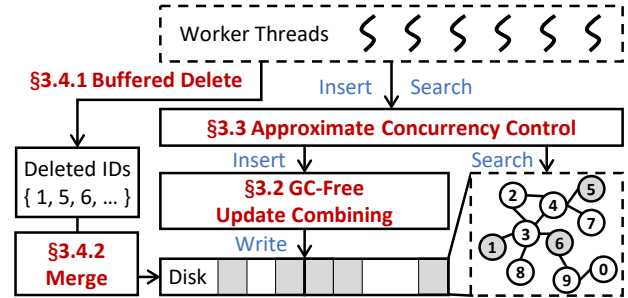


Figure 3: ODINANN overview.

- **High concurrency.** We leverage the approximate feature of operations to relax the isolation level of both insert and search for better concurrency (§3.3).

3.1 Overview

Figure 3 shows the overview of ODINANN.

Graph layout. The directed graph in ODINANN is organized as adjacent lists on disk (Figure 1), similar to previous works [24, 27]. Differently, ODINANN does space overprovision on disk for multiple free records on a page. Out-of-place record updates are combined in these pages (§3.2). In memory, PQ-compressed vectors are stored for graph navigation.

Operations. ODINANN supports three types of operations, vector search, insert, and delete.

- **Vector search.** ODINANN follows Algorithm 1 for vector search. Differently, ODINANN adopts a dynamic candidate pool, whose size increases with the number of deleted vectors in the pool, to ensure search quality in workloads containing deletes (§3.4.1). For better concurrency, ODINANN only ensures per-record consistency instead of the atomicity of the whole search (§3.3).
- **Direct insert.** ODINANN follows Algorithm 2 for direct vector insert. It updates the records out of place in the overprovisioned disk space, to combine them for less write amplification (§3.2). For concurrency control, ODINANN uses an approximate snapshot of the target vector’s neighbors during insert to reduce its critical section (§3.3).
- **Buffered delete.** ODINANN records the ID of deleted vectors in memory (§3.4.1) and merges delete to disk periodically (§3.4.2), similar to previous works [23, 30]. It has low runtime memory consumption (only for IDs) and little interference with frontend search tasks during the merge.

3.2 GC-Free Update Combining

In-place record updates suffer from massive disk writes, while the log-structured layout induces frequent garbage collection.